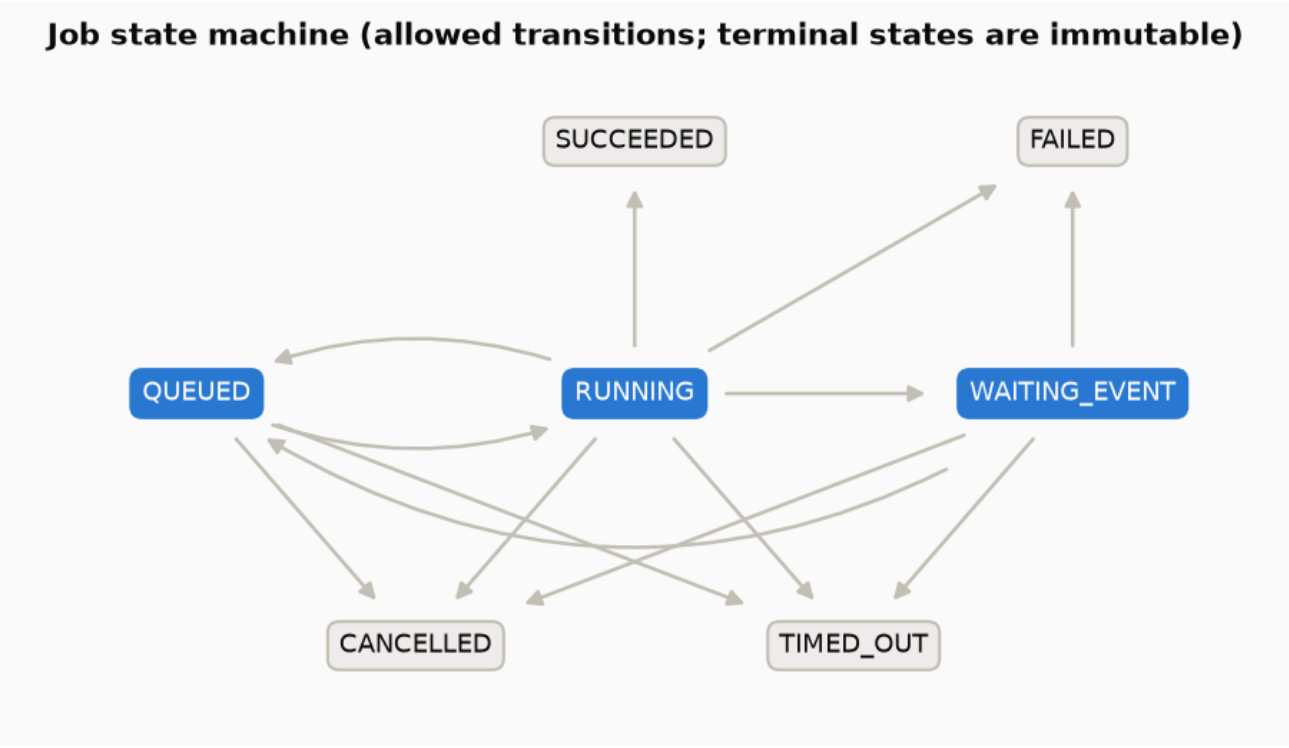
Отчёт: durable event-driven runtime для агента-составителя отчётов

Вариант стека: **custom runtime** — Python asyncio + SQLite (WAL) как durable state store + transactional outbox + таблицы `events`, `timers`, `scheduler`; FastAPI + CLI. Архитектура, форматы, таблица переходов, правило гонок и обоснование выбора — в README.md. **Все числа ниже вычислены скриптом `experiments/make_report.py` из сырых данных `results/raw/`** (симуляция), `results/real_llm/` (реальная LLM: Ollama, локально) и `results/real_llm_server/` (реальная LLM: сервер ITMO); разделы Simulated и два прогона Real LLM не смешиваются.

1. Окружение

параметр	значение
ОС	Windows-10-10.0.19045-SP0
CPU	AMD64 Family 25 Model 116 Stepping 1, AuthenticAMD (16 логич. ядер)
RAM	15.2 GB
Python	3.13.7
SQLite	3.50.4
fastapi	0.162.0
starlette	1.10.0
uvicorn	0.54.0
httpx	0.28.1
pydantic	2.14.3
pytest	9.2.0
matplotlib	3.11.2
numpy	2.5.4
Markdown	3.11.3
Масштаб времени, Part 1/2	1.0 (реальные секунды)
Масштаб времени, Part 3	0.1 → 1 virtual second = 0.1 real seconds
Seeds	Part 1: [1000, 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009]; Part 2: 7; Part 3: [11, 12, 13, 14, 15] (одинаковые для всех политик)
LLM в этих прогонах	**SimulatedLLM** (детерминированный); реальная LLM — раздел «Real LLM run»



2. Автотесты

Итог (Windows): **65 passed, 1 warning in 31.25s** (полный вывод `pytest -v`: `results/pytest_output.txt`).

Тот же набор в Linux-контейнере (`Dockerfile`, POSIX SIGKILL в тестах восстановления): **65 passed, 1 warning in 32.58s** (`results/raw_linux/pytest_output.txt`).

```
tests/test_cancel.py::test_cancel_queued PASSED                          [  1%]
tests/test_cancel.py::test_cancel_waiting PASSED                         [  3%]
tests/test_cancel.py::test_cancel_running_stops_step_and_no_new_steps PASSED [  4%]
```

```
tests/test_cancel.py::test_late_step_result_after_cancel_is_discarded PASSED [  6%]
tests/test_decide.py::test_llm_choice_is_applied_when_allowed PASSED     [  7%]
tests/test_decide.py::test_llm_cannot_skip_required_approval PASSED      [  9%]
tests/test_decide.py::test_llm_finish_allowed_without_approval PASSED    [ 10%]
tests/test_decide.py::test_llm_cannot_exceed_tool_budget PASSED          [ 12%]
tests/test_decide.py::test_unparseable_answer_falls_back_to_rules PASSED [ 13%]
tests/test_decide.py::test_rules_mode_does_not_call_llm PASSED           [ 15%]
tests/test_decide.py::test_llm_decide_end_to_end_with_simulated_llm[False] PASSED [ 16%]
tests/test_decide.py::test_llm_decide_end_to_end_with_simulated_llm[True] PASSED [ 18%]
tests/test_events.py::test_duplicate_event_applied_once PASSED           [ 20%]
tests/test_events.py::test_event_in_wrong_state_is_ignored_and_logged PASSED [ 21%]
tests/test_events.py::test_stale_approval_id_is_ignored PASSED           [ 23%]
tests/test_events.py::test_approval_after_cancel_or_timeout_does_not_change_state PASSED [ 24%]
tests/test_events.py::test_rejection_revises_then_fails PASSED           [ 26%]
tests/test_events.py::test_outbox_notifies_approval_request_and_terminal_state PASSED [ 27%]
tests/test_idempotency.py::test_create_job_same_key_returns_same_job PASSED [ 29%]
tests/test_idempotency.py::test_unique_index_is_the_backstop PASSED      [ 30%]
tests/test_idempotency.py::test_concurrent_creates_with_same_key_create_one_job PASSED [ 32%]
tests/test_idempotency.py::test_publisher_dedups_by_idempotency_key PASSED [ 33%]
tests/test_idempotency.py::test_publisher_ack_lost_then_retry_returns_previous_result PASSED [ 35%]
tests/test_idempotency.py::test_http_api_create_idempotent_and_429 PASSED [ 36%]
tests/test_idempotency.py::test_http_publisher_app PASSED                [ 38%]
tests/test_llm.py::test_simulated_llm_is_deterministic PASSED            [ 40%]
tests/test_llm.py::test_openai_compatible_success_and_usage PASSED       [ 41%]
tests/test_llm.py::test_openai_compatible_error_classification[429-TransientError] PASSED [ 43%]
tests/test_llm.py::test_openai_compatible_error_classification[503-TransientError] PASSED [ 44%]
tests/test_llm.py::test_openai_compatible_error_classification[400-PermanentError] PASSED [ 46%]
tests/test_llm.py::test_openai_compatible_error_classification[401-PermanentError] PASSED [ 47%]
tests/test_llm.py::test_openai_compatible_transport_error_is_transient PASSED [ 49%]
tests/test_races.py::test_deadline_beats_approval_when_timer_not_yet_fired PASSED [ 50%]
tests/test_races.py::test_cancel_beats_deadline_when_timer_not_yet_fired PASSED [ 52%]
tests/test_races.py::test_deadline_beats_step_completion PASSED          [ 53%]
tests/test_races.py::test_randomized_races_preserve_invariants PASSED    [ 55%]
tests/test_recovery.py::test_kill_between_steps_does_not_repeat_checkpointed_steps PASSED [ 56%]
tests/test_recovery.py::test_kill_during_llm_call_is_retried_after_lease_expiry PASSED [ 58%]
tests/test_recovery.py::test_publication_retracted_durably_when_job_cancelled_after_worker_crash PASSED [ 60%]
tests/test_recovery.py::test_side_effect_without_ack_is_not_duplicated PASSED [ 61%]
tests/test_retry.py::test_repeated_transient_error_bounded_then_failed PASSED [ 63%]
tests/test_retry.py::test_transient_then_success PASSED                  [ 64%]
tests/test_retry.py::test_permanent_error_fails_immediately PASSED       [ 66%]
tests/test_retry.py::test_token_budget_and_max_steps PASSED              [ 67%]
tests/test_retry.py::test_backoff_equal_jitter_bounds PASSED             [ 69%]
tests/test_retry.py::test_lease_loss_counts_as_attempt_poison_pill PASSED [ 70%]
tests/test_scheduling.py::test_global_and_per_tenant_limits_never_exceeded[fifo] PASSED [ 72%]
tests/test_scheduling.py::test_global_and_per_tenant_limits_never_exceeded[drr] PASSED [ 73%]
tests/test_scheduling.py::test_limits_hold_across_two_worker_processes_sharing_the_db PASSED [ 75%]
tests/test_scheduling.py::test_admission_control_rejects_with_reason PASSED [ 76%]
tests/test_scheduling.py::test_fifo_serves_heavy_backlog_first_drr_interleaves PASSED [ 78%]
tests/test_scheduling.py::test_aging_lets_old_low_priority_job_overtake PASSED [ 80%]
tests/test_smoke.py::test_happy_path_with_approval PASSED                [ 81%]
tests/test_states.py::test_allowed_transitions_table_matches_code PASSED [ 83%]
tests/test_states.py::test_terminal_row_cannot_be_updated_even_with_raw_sql[SUCCEEDED] PASSED [ 84%]
tests/test_states.py::test_terminal_row_cannot_be_updated_even_with_raw_sql[FAILED] PASSED [ 86%]
tests/test_states.py::test_terminal_row_cannot_be_updated_even_with_raw_sql[CANCELLED] PASSED [ 87%]
tests/test_states.py::test_terminal_row_cannot_be_updated_even_with_raw_sql[TIMED_OUT] PASSED [ 89%]
tests/test_states.py::test_invalid_transition_rejected_by_trigger PASSED [ 90%]
tests/test_states.py::test_state_change_requires_version_bump PASSED     [ 92%]
tests/test_states.py::test_cas_with_stale_version_fails PASSED           [ 93%]
tests/test_states.py::test_transition_log_written_by_trigger PASSED      [ 95%]
tests/test_timers_and_slots.py::test_deadline_in_queue_fires_after_restart PASSED [ 96%]
tests/test_timers_and_slots.py::test_deadline_while_waiting_survives_worker_restart PASSED [ 98%]
tests/test_timers_and_slots.py::test_waiting_job_does_not_occupy_execution_slot PASSED [100%]
```

Структурированные логи. Каждая запись — JSON с полями `job_id, step, attempt, event_id` (и `worker, tenant_id, state, data`). Пример (34 строк, один job с transient-ошибкой, дублем и поздним событием): `results/raw/sample_structured_log.jsonl`:

```
{"ts": 1794319839.33247, "level": "INFO", "logger": "agentrt.worker", "msg": "step start", "job_id": "job-41ee76c154704759", "step": "fetch_documents#0", "attempt": 1, "event_id": null, "worker": "w1:9b5cc863", "tenant_id": "tenant-demo", "state": null}
{"ts": 1794319839.348225, "level": "INFO", "logger": "agentrt.worker", "msg": "step start", "job_id": "job-41ee76c154704759", "step": "extract#0", "attempt": 1, "event_id": null, "worker": "w1:9b5cc863", "tenant_id": "tenant-demo", "state": null}
{"ts": 1794319839.364618, "level": "INFO", "logger": "agentrt.worker", "msg": "step start", "job_id": "job-41ee76c154704759", "step": "draft#0.0", "attempt": 1, "event_id": null, "worker": "w1:9b5cc863", "tenant_id": "tenant-demo", "state": null}
{"ts": 1794319839.393738, "level": "INFO", "logger": "agentrt.store", "msg": "transition", "job_id": "job-41ee76c154704759", "step": "draft#0.0", "attempt": 1, "event_id": null, "worker": null, "tenant_id": null, "state": "QUEUED", "data": {"src": "RUNNING", "dst": "QUEUED", "reason": "transient error at draft#0.0 (attempt 1): TransientError: simulated transient LLM error (503) on draft#0.0; retry in 0.024s"}}
{"ts": 1794319839.394028, "level": "WARNING", "logger": "agentrt.worker", "msg": "step transient error -> retry", "job_id": "job-41ee76c154704759", "step": "draft#0.0", "attempt": 1, "event_id": null, "worker": null, "tenant_id": null, "state": null, "data": {"error": "simulated transient LLM error (503) on draft#0.0"}}
{"ts": 1794319839.425472, "level": "INFO", "logger": "agentrt.worker", "msg": "step start", "job_id": "job-41ee76c154704759", "step": "draft#0.0", "attempt": 2, "event_id": null, "worker": "w1:9b5cc863", "tenant_id": "tenant-demo", "state": null}
```

3. Part 1 — восстановление и идемпотентность (Simulated)

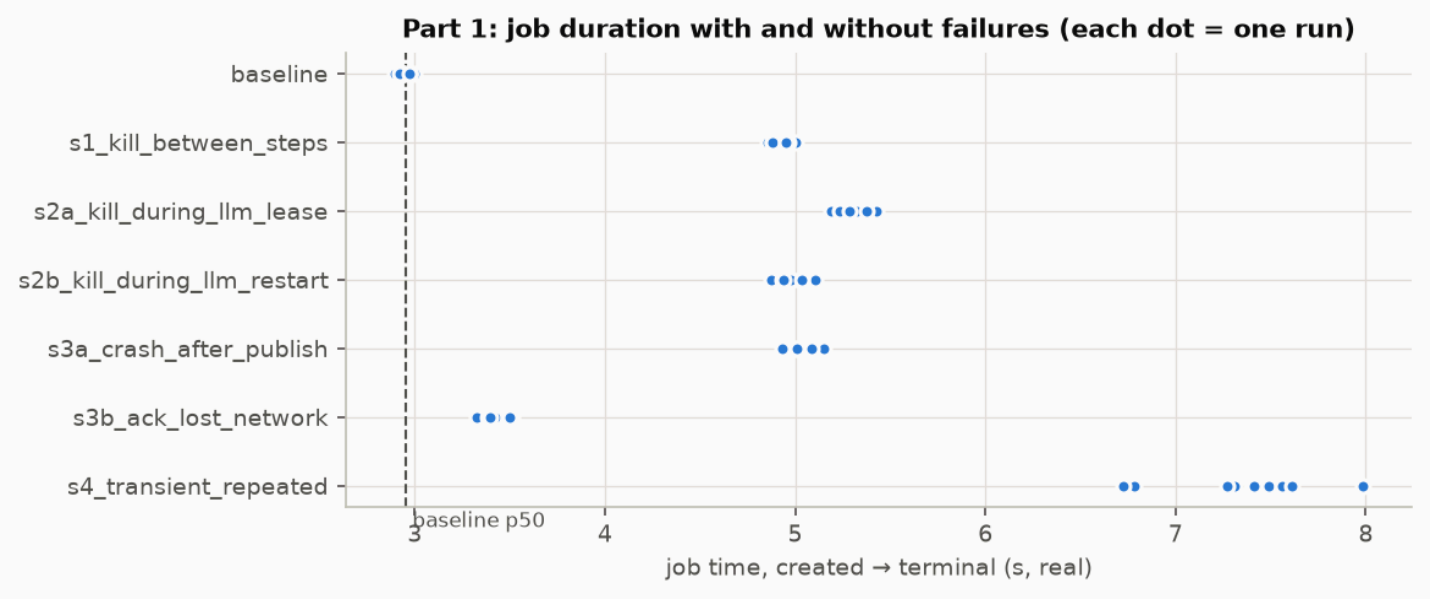
Время (реальные секунды; «возобновление» = первый шаг job после рестарта):

сценарий	N	итог	время job p50 [min–max]	+ к baseline p50	старт процесса → воркер готов p50	воркер готов → возобновление p50	старт процесса → возобновление p50
без отказов (baseline)	10	SUCCEEDED: 10	2.96 [2.89–3.00]	–	–	–	–
1. kill между шагами (после extract)	10	SUCCEEDED: 10	4.95 [4.85–5.00]	1.99	1.77	0.001	1.77
2a. kill во время LLM, другой воркер ждёт lease	10	SUCCEEDED: 10	5.35 [5.19–5.43]	2.39	1.73	0.409	2.09
2b. kill во время LLM, рестарт того же воркера	10	SUCCEEDED: 10	4.96 [4.87–5.11]	2.00	1.71	0.022	1.71
3a. крэш после publish (ack не сохранён)	10	SUCCEEDED: 10	5.00 [4.93–5.15]	2.05	1.81	0.022	1.83
3b. ack потерян в сети (503 после коммита)	10	SUCCEEDED: 10	3.41 [3.32–3.51]	0.45	–	–	–
4. повторяющаяся transient-ошибка	10	FAILED: 10	7.41 [6.72–7.99]	4.45	–	–	–

Повторы и побочные эффекты (суммы по N прогонам):

сценарий	LLM-вызовов	повторных LLM	повторных tool	вызовов с неизвестным исходом	перевыполнено шагов с чекпоинтом	publish физ. / лог.	дублей лог. публикаций	доп. токены прерванных вызовов: подтверждено / верхняя граница	доп. стоимость USD: подтверждено / верх.
без отказов (baseline)	20	0	0	0	0	10 / 10	0	0 / ≤0	0.000000 / ≤0.000000
1. kill между шагами (после extract)	20	0	0	0	0	10 / 10	0	0 / ≤0	0.000000 / ≤0.000000
2a. kill во время LLM, другой воркер ждёт lease	30	10	0	10	0	10 / 10	0	0 / ≤2994	0.000000 / ≤0.001349
2b. kill во время LLM, рестарт того же воркера	30	10	0	10	0	10 / 10	0	0 / ≤2994	0.000000 / ≤0.001349
3a. крэш после publish (ack не сохранён)	20	0	0	0	0	20 / 10	0	0 / ≤0	0.000000 / ≤0.000000
3b. ack потерян в сети (503 после коммита)	20	0	0	0	0	20 / 10	0	0 / ≤0	0.000000 / ≤0.000000
4. повторяющаяся transient-ошибка	40	30	0	0	0	0 / 0	0	0 / ≤0	0.000000 / ≤0.000000

Конфигурация: lease_timeout = 2.0 с, heartbeat = 0.25 с, max_attempts = 4, backoff base = 0.5 с, cap = 4.0 с; по 10 повторов на сценарий; воркер — отдельный процесс, убивается `Popen.kill()` (на Windows это `TerminateProcess` — аналог SIGKILL: без обработчиков и `finally`), FakePublisherService — отдельный HTTP-процесс. Сырые данные: `results/raw/recovery_runs.csv`, `recovery_step_runs.jsonl`, `recovery_calls.jsonl`.



Наблюдения (из таблицы):

- **Kill между шагами.** Повторных LLM/tool вызовов: 0/0, перевыполненных шагов с чекпоинтом: 0. Шаги `fetch_documents` / `extract`, чей результат уже в `step_results`, не повторялись. Возобновление после того, как новый процесс готов: p50 = 0.001 с (тот же `--name` ⇒ lease прежней инкарнации забирается сразу при старте); основная часть времени «от старта процесса» (p50 1.77 с) — запуск интерпретатора и импорт модулей.
- **Kill во время LLM-вызова.** Каждый прерванный вызов повторён ровно один раз (повторных LLM: 10 на 10 прогонов в 2a и 10 на 10 в 2b). Если подхватывает *другой* воркер, он ждёт истечения lease: возобновление через 0.41 с после его готовности (остаток lease_timeout), против 0.022 с при рестарте того же воркера. **Неопределённость исхода:** 20 вызовов остались в журнале вызовов без завершения — рантайм не знает, обработал

ли их провайдер и выставил ли счёт. Поэтому дополнительные токены даны интервалом: фактически оплаченные повторы, которые мы видим, = 0, верхняя граница (prompt + max_output_tokens прерванных вызовов) = 5988 токенов на 20 прогонов.

- **Side effect без ack.** В 3a процесс умирает сразу после ответа сервиса (до чекпоинта), в 3b сервис коммитит и отвечает 503. В обоих случаях физических запросов публикации 2 на job, логических — 1, дублей логических публикаций 0: повтор идёт с тем же `Idempotency-Key`, и сервис возвращает прежний `publication_id` (`replayed=true`).
- **Повторяющаяся transient-ошибка.** Ровно 4 попытки на прогон (всего 40 неудачных попыток в 10 прогонах), паузы между попытками (с, по прогонам): [0.28, 0.77, 1.68]; [0.34, 0.70, 1.51]; [0.27, 0.55, 1.96]; [0.33, 0.68, 1.64]; [0.50, 0.76, 1.98]; [0.48, 0.81, 1.37]; [0.33, 0.77, 1.82]; [0.34, 0.56, 1.17]; [0.41, 0.77, 1.36]; [0.26, 0.57, 1.14] — растут экспоненциально с jitter (ожидаемые интервалы [0.25–0.5], [0.5–1.0], [1.0–2.0] плюс время поллинга). Итог — `FAILED` с причиной: `step draft#0.0 failed after 4 attempts (max_attempts=4); last error: TransientError: simulated transient LLM error (503) on draft#0.0`.

Part 1 на Linux: тот же эксперимент в Docker-контейнере (настоящий SIGKILL)

Окружение: Linux-6.18.58.1-microsoft-standard-WSL2-x86_64-with-glibc2.41, Python 3.13.18 (образ из `Dockerfile`, Docker Desktop на этой же машине, ядро WSL2). Код и параметры те же (`python -m experiments.exp_recovery --reps 10`), но `Popen.kill()` здесь посылает POSIX SIGKILL. Сырые данные — отдельно, в `results/raw_linux/`; с Windows-прогоном выше не смешиваются. Команда: `.\scripts\run_linux.ps1 -Parts recovery`.

сценарий	N	итог	время job p50 [min–max], с	воркер готов → возобновление p50, с	повторных LLM	перевыполнено шагов с чекпоинтом	publish физ. / лог.	дублей лог. публикаций
без отказов (baseline)	10	SUCCEEDED: 10	2.78 [2.77–2.83]	–	0	0	10 / 10	0
1. kill между шагами (после extract)	10	SUCCEEDED: 10	3.72 [3.65–3.80]	0.000	0	0	10 / 10	0
2a. kill во время LLM, другой воркер ждёт lease	10	SUCCEEDED: 10	5.12 [5.06–5.23]	1.308	10	0	10 / 10	0
2b. kill во время LLM, рестарт того же воркера	10	SUCCEEDED: 10	3.86 [3.80–3.89]	0.000	10	0	10 / 10	0
3a. крэш после publish (ack не сохранён)	10	SUCCEEDED: 10	3.74 [3.72–3.78]	0.000	0	0	20 / 10	0
3b. ack потерян в сети (503 после коммита)	10	SUCCEEDED: 10	3.33 [3.18–3.40]	–	0	0	20 / 10	0
4. повторяющаяся transient-ошибка	10	FAILED: 10	7.23 [6.49–7.78]	–	30	0	0 / 0	0

Гарантии воспроизвелись и под SIGKILL: шаги с чекпоинтом не перевыполняются, прерванный LLM-вызов повторяется ровно один раз, повторная публикация не создаёт дубля, transient-ошибка исчерпывает 4 попытки и даёт `FAILED`. Абсолютные времена отличаются от Windows (другой запуск процесса, файловая система тома). В 2a «готов → возобновление» — это остаток lease (2.0 с от последнего heartbeat), который новый воркер ещё ждёт после своей готовности: на Linux процесс готов быстрее (старт → готов p50 0.74 с против 1.73 с на Windows), поэтому ждать остаётся дольше (1.31 против 0.41 с), а время от старта процесса до возобновления близко (2.02 против 2.09 с).

4. Part 2 — событийный жизненный цикл (Simulated)

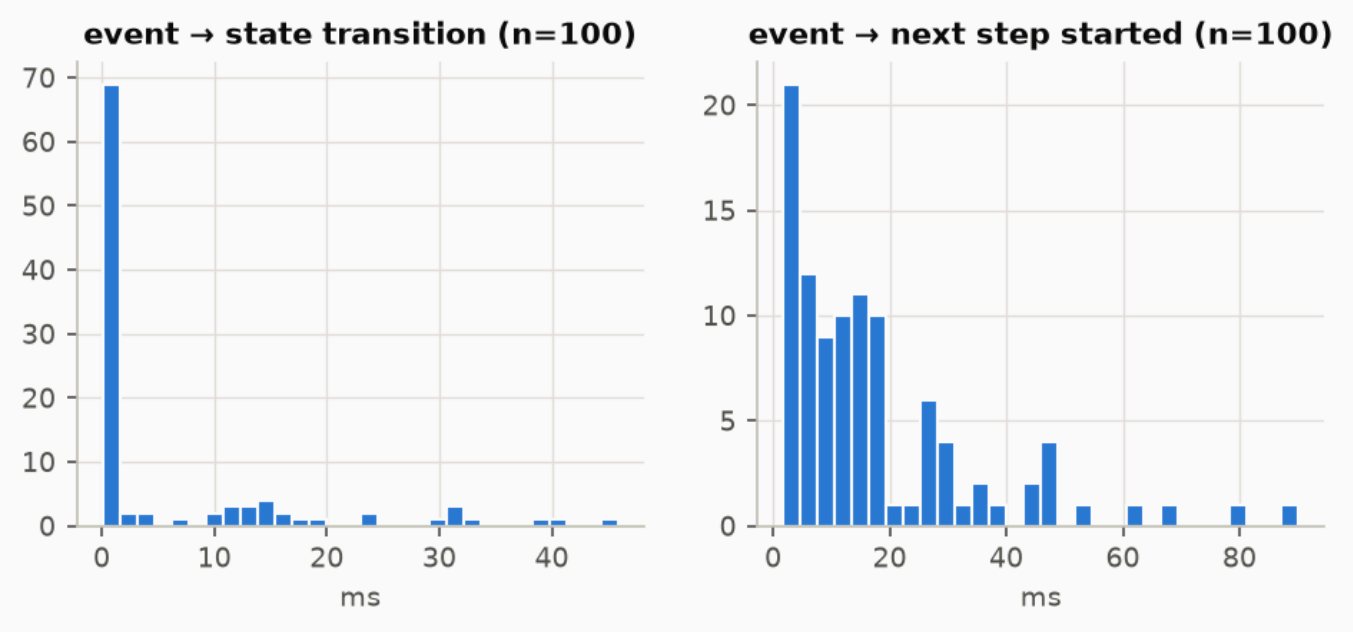
Все эксперименты — `time_scale = 1.0`, воркер в отдельном потоке (E1–E5, E7) или отдельном процессе (E6). Латентности измерены по меткам времени, записанным в БД в момент коммита (`events.received_at`, `transitions.ts`, `step_runs.started_at`). Сырые данные: `results/raw/events_*.csv`, `events_summary.json`.

E1. Approval вовремя

100 job, итог: SUCCEEDED: 100; исходы событий: applied: 100.

метрика	N	p50, мс	p95, мс	max, мс
event_to_transition_ms	100	0.16	32.01	45.89
client_roundtrip_ms	100	0.90	32.53	46.74
event_to_next_step_start_ms	100	12.82	48.50	90.18

«event → state transition» — от приёма события до коммита перехода `WAITING_EVENT → QUEUED` (inbox и переход в одной транзакции). «event → next step started» дополнительно включает ожидание цикла планировщика (поллинг `poll_interval_s = 3 мс`, межпроцессного уведомления нет). Медиана — доли миллисекунды, а хвост (p95/max) — по всей видимости ожидание блокировки записи: если в этот момент транзакцию держит воркер, `BEGIN IMMEDIATE` отправителя попадает в busy-handler SQLite, который ждёт ступенями (1, 2, 5, 10, 15, 20, 25… мс). Это плата за single-writer хранилище.



E2. Approval не пришёл → TIMED_OUT

40 job с дедлайном 0.5 с: итог TIMED_OUT: 40. Запаздывание перехода относительно дедлайна: p50 = 4.074 мс, p95 = 13.33 мс, max = 14.136 мс (N = 40). Шагов, исполнявшихся во время ожидания: 0.

E3. Дубли события

Отправлено 250 копий approval (на каждую job — 3 одновременно из разных потоков и 2 повторно позже, все с одним `event_id`): применено 50, распознано как дубли 200; job с более чем одним переходом `WAITING_EVENT → QUEUED`: 0.

E4. Approval после CANCELLED / TIMED_OUT

40 поздних событий: исходы ignored_terminal: 40; повторный cancel: already_terminal: 40; изменилось состояние: 0, изменилась версия строки: 0.

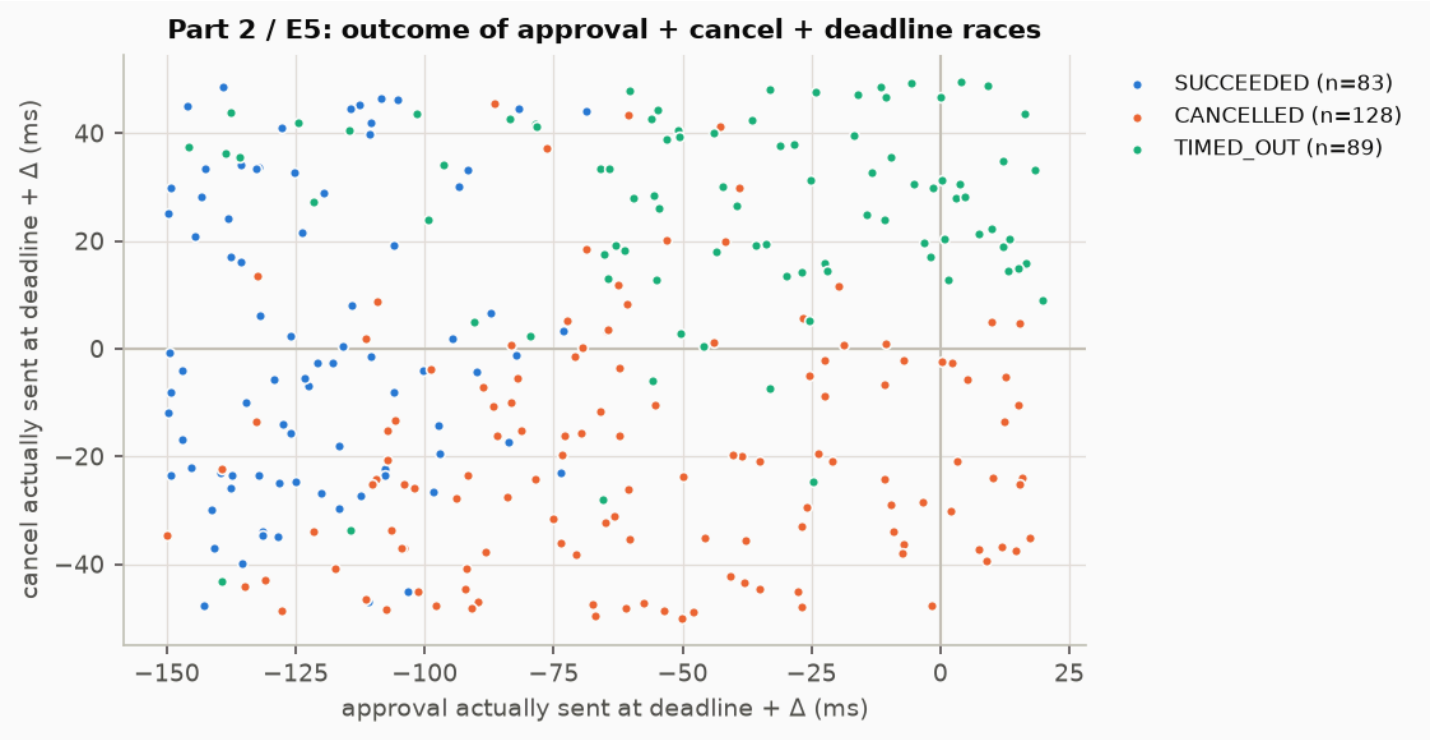
E5. Почти одновременные approval + cancel + deadline + завершение шагов

300 повторов. Для каждой job дедлайн T = создание + 0.6 с; approval отправляется в T + U(−150, +20) мс, cancel — в T + U(−60, +60) мс (seed 7); после approval воркер исполняет `save_report` и `publish` (~30–50 мс), так что их завершение тоже попадает в окно гонки. Плановые сдвиги и фактические моменты отправки (`approve_sent_ms`, `cancel_sent_ms` — по часам отправителя перед вызовом) записаны в `events_e5_races.csv`; на графике — фактические (они могут отличаться от плановых из-за планирования потоков ОС и ожидания блокировки записи; если job ещё не дошла до `WAITING_EVENT`, отправитель ждёт её, поэтому часть событий уходит позже дедлайна).

итоговое состояние	N
CANCELLED	128
SUCCEEDED	83
TIMED_OUT	89

Откуда пришли в терминал: QUEUED->CANCELLED: 8, QUEUED->TIMED_OUT: 14, RUNNING->CANCELLED: 82, RUNNING->SUCCEEDED: 83, RUNNING->TIMED_OUT: 46, WAITING_EVENT->CANCELLED: 38, WAITING_EVENT->TIMED_OUT: 29. Исходы approval: applied: 233, deadline_won: 12, ignored_terminal: 55. Исходы cancel: already_terminal: 172, cancelled: 128. Счётчики рантайма: cancel_after_terminal: 172, events_late: 55, late_results_discarded: 55.

Инварианты: job с ≠1 терминальным переходом — 0; публикаций, принадлежащих не-SUCCEEDED job и не отозванных — 0; SUCCEEDED без живой публикации — 0; публикаций, отозванных компенсацией — 64; попыток недопустимого перехода, отклонённых триггером — 0.



Проверка правила гонок по каждой job (из `events_e5_races.csv`, метки — моменты коммита в БД; N = 300):

что было бы нарушением правила	job
SUCCEEDED закоммичен не раньше дедлайна (запрещено: шаг проверяет now ≥ deadline)	0
TIMED_OUT раньше дедлайна	0
CANCELLED без успешного cancel	0
cancel успешен, а итог не CANCELLED	0
approval `deadline_won`, а итог не TIMED_OUT	0
approval применён, хотя отправлен после дедлайна	0
больше одного терминального перехода	0

Всего нарушений: 0. Отдельно: 31 job получили CANCELLED позже дедлайна (до 45.84 мс) — это допустимо по правилу: cancel не проверяет дедлайн и побеждает, если закоммичен раньше, чем таймер дедлайна (поллинг) успел перевести job в TIMED_OUT; первая закоммиченная терминальная транзакция выигрывает.

Картина соответствует правилу: cancel, закоммиченный раньше дедлайна и раньше завершения `publish`, даёт CANCELLED; если approval пришёл достаточно рано, шаги успевают закоммитить SUCCEEDED, и более поздние cancel получают `already_terminal`; если к моменту approval или коммита шага дедлайн уже прошёл, побеждает TIMED_OUT (`deadline_won`, `aborted_deadline`, `late_results_discarded`).

Воспроизводимость E5. Seed фиксирует только *плановые* моменты отправки; фактический порядок коммитов зависит от планирования потоков ОС и ожидания блокировки записи, поэтому распределение итоговых состояний (CANCELLED / SUCCEEDED / TIMED_OUT) от запуска к запуску заметно меняется (наблюдалось при контрольных перезапусках; их сырые данные в репозиторий не входят). Воспроизводимым результатом E5 являются инварианты (0 нарушений во всех прогонах) и соответствие каждого исхода правилу гонок, а не доли исходов.

Исправление стенда E5 (найдено при прогоне на Linux). В первой версии каждый из 20 потоков-отправителей сам опрашивал БД каждые 3 мс в том же процессе, что и воркер. На Windows это почти не влияло, а в Linux-контейнере из-за конкуренции за GIL воркер вытеснялся, job не успевали дойти до ожидания за 0.6 с, и все 300 повторов заканчивались TIMED_OUT — гонки не было (сырые данные того прогона перезаписаны исправленным; эффект воспроизводится диагностикой ниже). Диагностика `experiments/diag_e5_polling.py` (10 job, p50 пути «создание → `WAITING_EVENT`»): Linux — 7.21 с с 20 опрашивающими потоками против 0.14 с без них; Windows — 0.23 против 0.19 с (`results/raw_linux/diag_e5_polling.json`, `results/raw/diag_e5_polling.json`). Сейчас состояние опрашивает один поток-монитор, отправители ждут его сигнала; весь Part 2 перезапущен этим кодом на обеих ОС — данные выше и в блоке «на Linux» уже с исправленным стендом. Рантайм не менялся: это свойство стенда, где воркер и отправители живут в одном процессе.

E6. Рестарт воркера во время ожидания

вариант	повтор	итог	открытых step_runs в ожидании	состояние пока воркер мёртв	исход события	готов → первый шаг, мс	готов → терминал, мс	опоздание таймаута, мс
approval_while_down	0	SUCCEEDED	0	QUEUED	applied	0.49	66.53	–
deadline_while_down	0	TIMED_OUT	0	WAITING_EVENT	–	–	-1.49	1217.56
approval_after_restart	0	SUCCEEDED	0	WAITING_EVENT	applied	7.82	66.06	–
approval_while_down	1	SUCCEEDED	0	QUEUED	applied	0.49	52.69	–
deadline_while_down	1	TIMED_OUT	0	WAITING_EVENT	–	–	-1.22	1218.91
approval_after_restart	1	SUCCEEDED	0	WAITING_EVENT	applied	7.66	64.49	–

вариант	повтор	итог	открытых step_runs в ожидании	состояние пока воркер мёртв	исход события	готов → первый шаг, мс	готов → терминал, мс	опоздание таймаута, мс
approval_while_down	2	SUCCEEDED	0	QUEUED	applied	0.60	58.76	–
deadline_while_down	2	TIMED_OUT	0	WAITING_EVENT	–	–	-1.31	1210.25
approval_after_restart	2	SUCCEEDED	0	WAITING_EVENT	applied	8.34	59.58	–
approval_while_down	3	SUCCEEDED	0	QUEUED	applied	0.53	58.02	–
deadline_while_down	3	TIMED_OUT	0	WAITING_EVENT	–	–	-1.26	1182.62
approval_after_restart	3	SUCCEEDED	0	WAITING_EVENT	applied	21.54	71.75	–
approval_while_down	4	SUCCEEDED	0	QUEUED	applied	0.58	62.02	–
deadline_while_down	4	TIMED_OUT	0	WAITING_EVENT	–	–	-1.35	1209.58
approval_after_restart	4	SUCCEEDED	0	WAITING_EVENT	applied	13.94	63.50	–

Ожидание не держит слот (0 открытых `step_runs`), событие принимается и применяется даже когда воркер мёртв (inbox в БД), а дедлайн, истёкший во время простоя, срабатывает при старте нового воркера (таймер хранится в БД). Для `deadline_while_down` «опоздание таймаута» ≈ длительность простоя — таймер не может сработать, пока нет ни одного живого воркера; отрицательное «готов → терминал» означает, что просроченный таймер сработал в процедуре старта воркера ещё до отметки готовности.

E7. Время до реальной остановки после cancel

отмена в состоянии	N	p50, мс	p95, мс	max, мс (QUEUED/WAITING: 0 по определению)
QUEUED	15	0.00	0.00	0.00
RUNNING	15	70.28	101.60	107.27
WAITING_EVENT	15	0.00	0.00	0.00

Исходы LLM-вызовов в E7: cancelled: 15, ok: 15. В `RUNNING` остановка ограничена интервалом heartbeat (0.1 с в E7): шаг узнаёт об отмене на следующем heartbeat и прерывает LLM-вызов; в `QUEUED` и `WAITING_EVENT` исполняемой работы нет — остановка мгновенная (коммит перехода).

Сводка инвариантов (переходы из журнала `transitions`, пишется триггером)

эксперимент	проверено переходов	недопустимых переходов	изменений терминального статуса	разрывов версии	job с >1 терминальным переходом	журнал ≠ состояние
E1	1400	0	0	0	0	0
E2	400	0	0	0	0	0
E3	700	0	0	0	0	0
E4	400	0	0	0	0	0
E5	3778	0	0	0	0	0
E7	285	0	0	0	0	0

Part 2 на Linux (Docker-контейнер)

Окружение: Linux-6.18.58.1-microsoft-standard-WSL2-x86_64-with-glibc2.41, Python 3.13.18; те же скрипт и параметры (`exp_events --race-reps 300`, seed 7). Сырые данные отдельно — `results/raw_linux/events_*`; с Windows-прогоном выше не смешиваются (команда: `.\scripts\run_linux.ps1`).

эксперимент	результат на Linux
E1 approval вовремя	SUCCEEDED: 100; event → transition p50 0.43 / p95 8.981 мс (N = 100)
E2 approval не пришёл	TIMED_OUT: 40; опоздание таймаута p50 3.54 / p95 6.789 мс (N = 40); шагов во время ожидания: 0
E3 дубли события	отправлено 250, применено 50, дублей 200; job с >1 переходом: 0
E4 approval после терминала	40 поздних: ignored_terminal: 40; изменений состояния: 0
E5 гонки	CANCELLED: 63, SUCCEEDED: 175, TIMED_OUT: 62; job с ≠1 терминальным переходом: 0; неотозванных публикаций не-SUCCEEDED job: 0; нарушений правила гонок по каждой job: 0; CANCELLED позже дедлайна (допустимо): 4
E6 рестарт во время ожидания	15 из 15 прогонов с ожидаемым итогом и 0 открытых step_runs
E7 остановка после cancel в RUNNING	p50 55.16 / p95 76.224 мс (N = 15)
инварианты E1–E5, E7	проверено переходов 7072, нарушений 0

Поведение совпадает с Windows по всем проверкам (итоги E1–E4, E6, нулевые нарушения инвариантов). Латентности и доли исходов E5 различаются — они зависят от планировщика ОС и файловой системы тома (см. «Воспроизводимость E5»).

5. Part 3 — backpressure и честное планирование (Simulated)

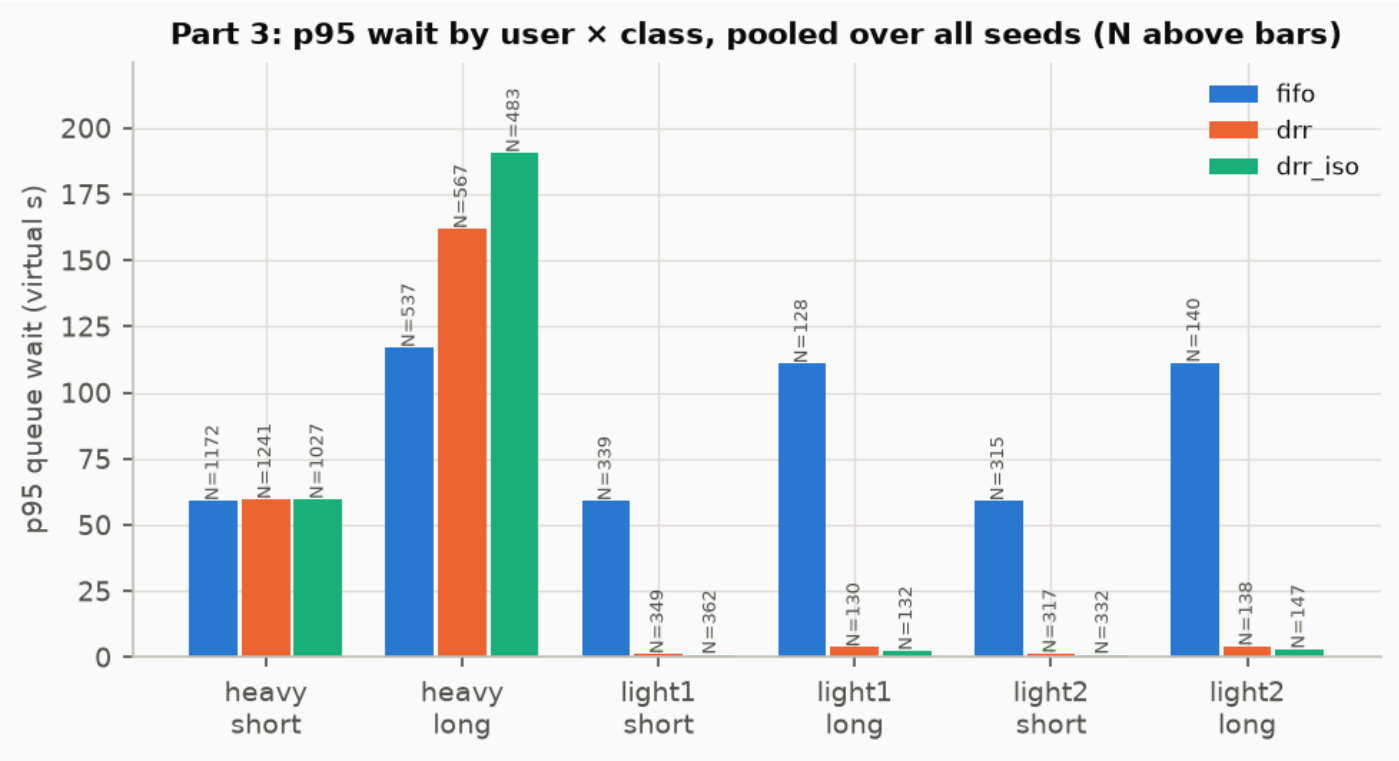
Нагрузка. 3 пользователя (`heavy` шлёт сильно больше остальных), 2 класса: short ≈ 1.00 с и long ≈ 9.50 с номинальной работы (long = 30% заявок; long разбит на шаги ≤ 1.5 с), 10% заявок с priority = 1. Фазы (заявок в вирт. секунду): P1_below [0–150 с): heavy 0.5/с, light1 0.2/с, light2 0.2/с; P2_burst [150–240 с): heavy 3.0/с, light1 0.2/с, light2 0.2/с; P3_recovery [240–480 с): heavy 0.3/с, light1 0.2/с, light2 0.2/с. Пуассоновские прибытия, трасса генерируется из seed и сохраняется (`results/raw/load_arrivals_seed*.csv`) — **один и тот же поток заявок для всех политик**. Ресурсы одинаковые: `global_concurrency = 6` (номинальная ёмкость ≈ 1.69 job/с), `max_queue_size = 150` (admitted QUEUED+RUNNING), дедлайны short 60 с / long 200 с. **Масштаб времени: 1 вирт. с = 0.1 реальной с** (все времена ниже — виртуальные секунды). Seeds [11, 12, 13, 14, 15], по одному прогону каждой политики на seed.

Политики: * `fifo` — общая очередь по `ready_at`, без лимита на tenant (`per_tenant_concurrency = global`); * `drr` — deficit round robin по tenant (веса 1:1:1, quantum 1 вирт. с работы) + aging приоритетов (+1 за 10 с ожидания), без жёсткого лимита на tenant (work-conserving); * `drr_iso` — `drr` +

`per_tenant_concurrency = 3` + квота очереди `max_queue_per_tenant = 100` (жёсткая изоляция).

Ожидание по user × class (ожидание = суммарное время в очереди в состоянии «готов к запуску», по всем шагам; для истёкших в очереди — до момента истечения)

политика	user	class	N	p50 ожидания, с	p95 ожидания, с	p95 по seed (min–max)	max, с	истекло (TIMED_OUT)	e2e p50 успешных, с
fifo	heavy	short	1172	42.41	59.12	59.10–59.15	59.49	502	1.64
fifo	heavy	long	537	74.84	117.01	97.33–118.50	119.37	0	84.82
fifo	light1	short	339	0.12	59.08	59.03–59.10	59.15	46	1.38
fifo	light1	long	128	0.41	110.94	94.65–114.84	117.88	0	10.30
fifo	light2	short	315	0.15	59.05	59.00–59.09	59.12	51	1.38
fifo	light2	long	140	2.11	111.29	74.98–115.42	117.54	0	11.96
drr	heavy	short	1241	58.92	59.66	59.17–59.67	59.81	657	1.47
drr	heavy	long	567	98.29	161.93	127.65–166.71	168.82	0	107.93
drr	light1	short	349	0.14	1.24	0.84–1.71	4.61	0	1.44
drr	light1	long	130	0.79	3.79	2.56–4.56	4.91	0	10.63
drr	light2	short	317	0.13	1.47	0.93–2.56	5.72	0	1.46
drr	light2	long	138	0.95	4.20	2.00–6.32	6.72	0	10.86
drr_iso	heavy	short	1027	59.05	59.67	59.66–59.68	60.07	573	1.72
drr_iso	heavy	long	483	117.09	190.62	152.99–190.88	191.11	38	115.63
drr_iso	light1	short	362	0.04	0.98	0.39–2.56	4.87	0	1.33
drr_iso	light1	long	132	0.15	2.49	0.85–3.66	4.70	0	10.20
drr_iso	light2	short	332	0.04	1.01	0.65–2.61	7.42	0	1.32
drr_iso	light2	long	147	0.09	3.13	0.71–5.92	6.42	0	10.15



Ожидание по приоритету (aging и защита от голодания; все 5 seed)

политика	пользователи	priority	N	p50 ожидания, с	p95 ожидания, с	max, с	SUCCEEDED	TIMED_OUT
fifo	heavy	0	1557	49.26	110.99	119.37	1103	454
fifo	heavy	1	152	49.40	113.75	118.14	104	48
fifo	light1+light2	0	856	0.34	86.68	117.88	767	89
fifo	light1+light2	1	66	0.07	62.73	117.54	58	8
drr	heavy	0	1644	59.04	147.16	168.82	999	645
drr	heavy	1	164	1.72	73.44	105.86	152	12
drr	light1+light2	0	868	0.25	2.77	6.72	868	0
drr	light1+light2	1	66	0.08	1.64	2.04	66	0
drr_iso	heavy	0	1378	59.11	183.65	191.11	789	589
drr_iso	heavy	1	132	1.41	110.19	128.03	110	22

политика	пользователи	priority	N	p50 ожидания, с	p95 ожидания, с	max, с	SUCCEEDED	TIMED_OUT
drr_iso	light1+light2	0	902	0.05	2.09	7.42	902	0
drr_iso	light1+light2	1	71	0.04	0.73	1.09	71	0

В `fifo` приоритет не учитывается: у `heavy` p50 ожидания priority 0 и 1 почти одинаковы (49.26 и 49.40 с). В `drr` priority = 1 внутри tenant обслуживается первым (p50 1.72 с против 59.04 с у priority 0), но задачи priority 0 не вытесняются бесконечно: aging (+1 к эффективному приоритету за 10 с ожидания) возвращает их в оборот — 999 из 1644 завершились успешно. Остальные (645) истекли по дедлайну: при длительной перегрузке «защита от голодания» не создаёт ёмкость, и дедлайн снимает в первую очередь давно ждущую низкоприоритетную работу перегружающего tenant. У лёгких пользователей приоритет почти не влияет — их спрос ниже доли. Для priority = 1 наблюдений мало (10 % заявок), см. столбец N.

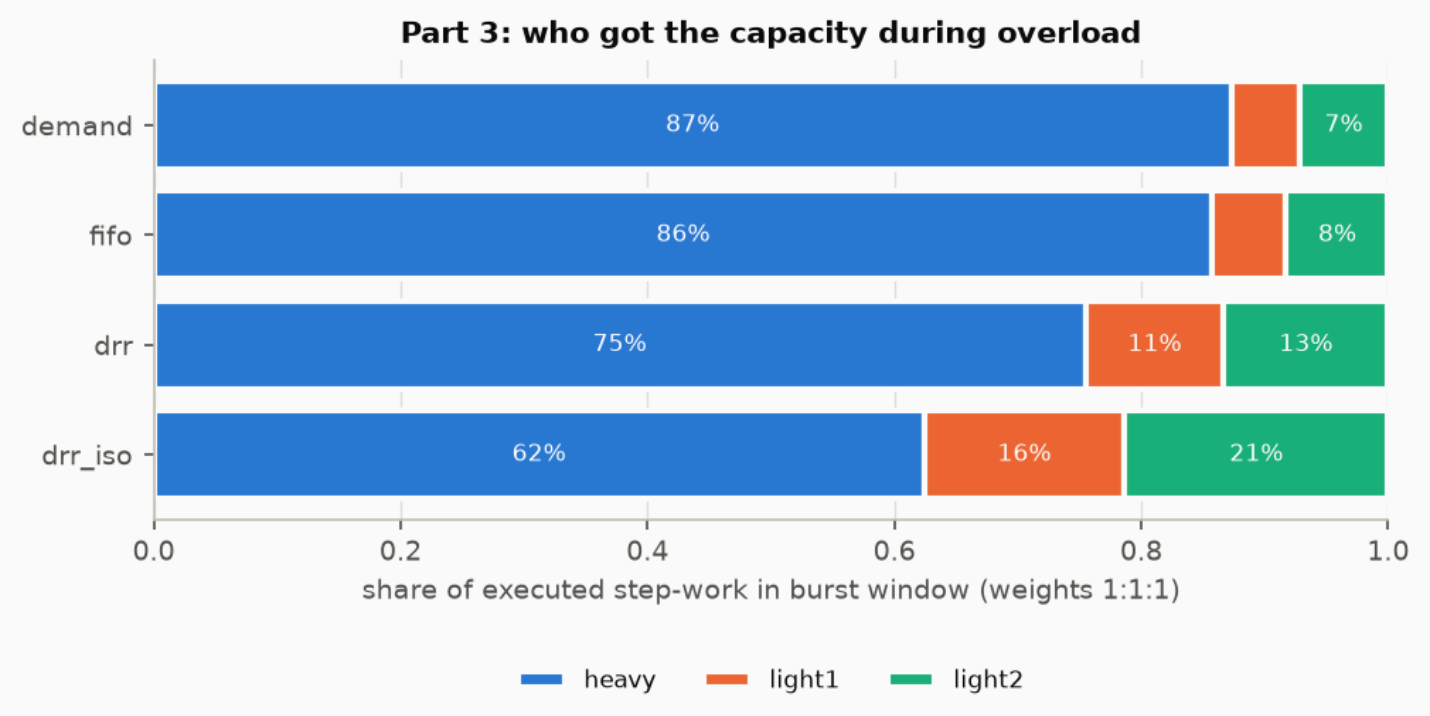
Сводка по политикам (сумма/максимум по 5 прогонам на политику)

политика	прогонов	заявок	принято	отклонено (по user)	SUCCEEDED	TIMED_OUT (по user)	jobs/s (весь прогон)	jobs/s в burst	загрузка слотов в burst	доля слот-времени на job, которые потом истекли	max суммарное ожидание job, с	max возраст старейшей job в очереди (от создания), с	max ожидание старейшего готового шага (от готовности), с	max длина очереди	max одновременно global / лимит	max на tena / лим
fifo	5	2994	2631	363 (heavy: 312, light1: 27, light2: 24)	2032	599 (heavy: 502, light1: 46, light2: 51)	0.839	0.704	98.3%	6.0%	119.37	128.48	16.53	146	6 / 6	6 / 6
drr	5	2994	2742	252 (heavy: 213, light1: 15, light2: 24)	2085	657 (heavy: 657)	0.861	0.793	98.2%	5.3%	168.82	178.23	32.81	146	6 / 6	6 / 6
drr_iso	5	2994	2483	511 (heavy: 511)	1872	611 (heavy: 611)	0.773	0.656	77.5%	7.8%	191.11	200.00	32.27	99	6 / 6	3 / 3

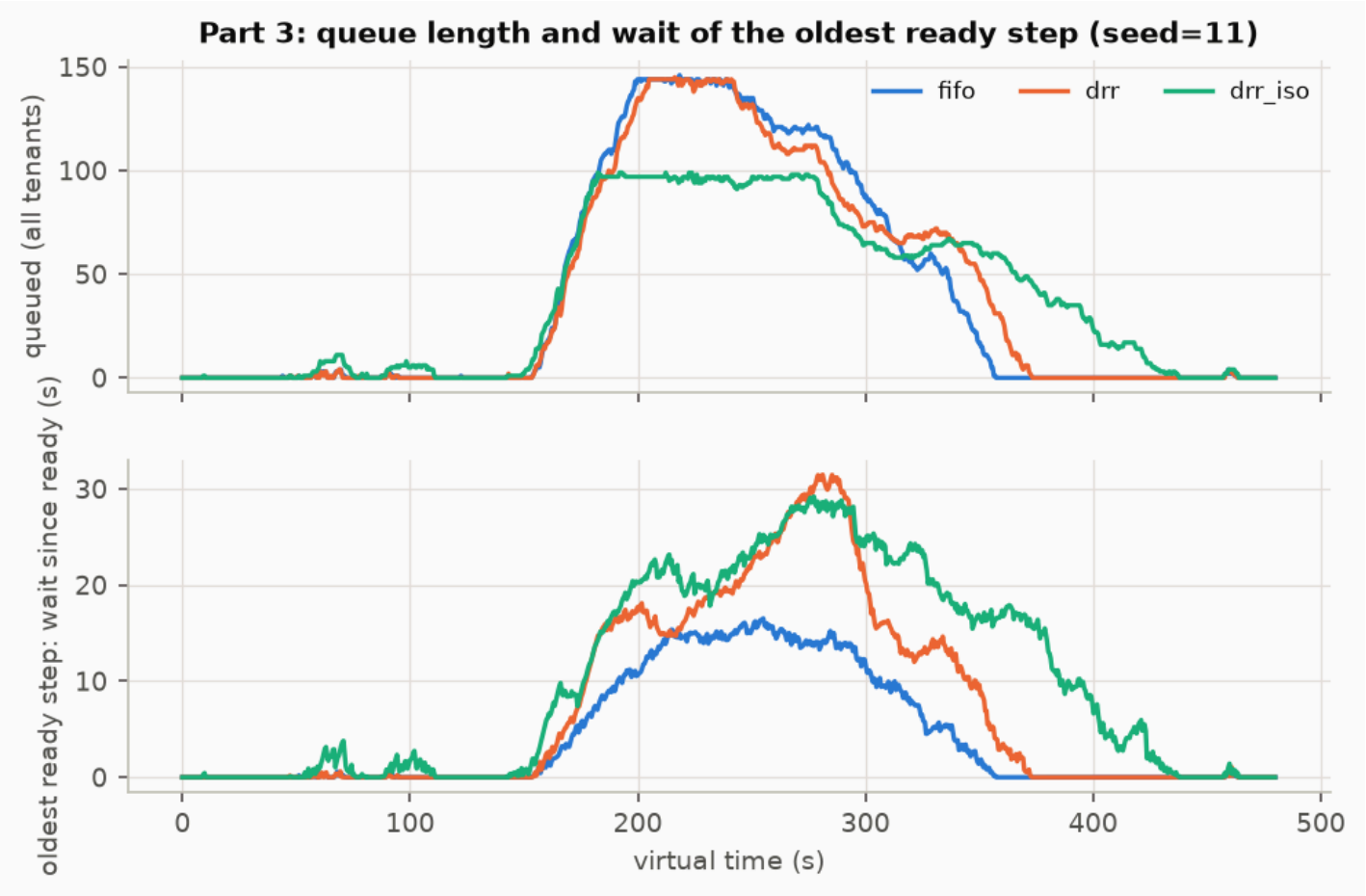
Масштабирование времени добавляет накладные расходы: медиана (фактическая − номинальная длительность шага) = 0.029 вирт. с, p95 = 0.165 вирт. с (транзакции SQLite и цикл asyncio в реальном времени не масштабируются).

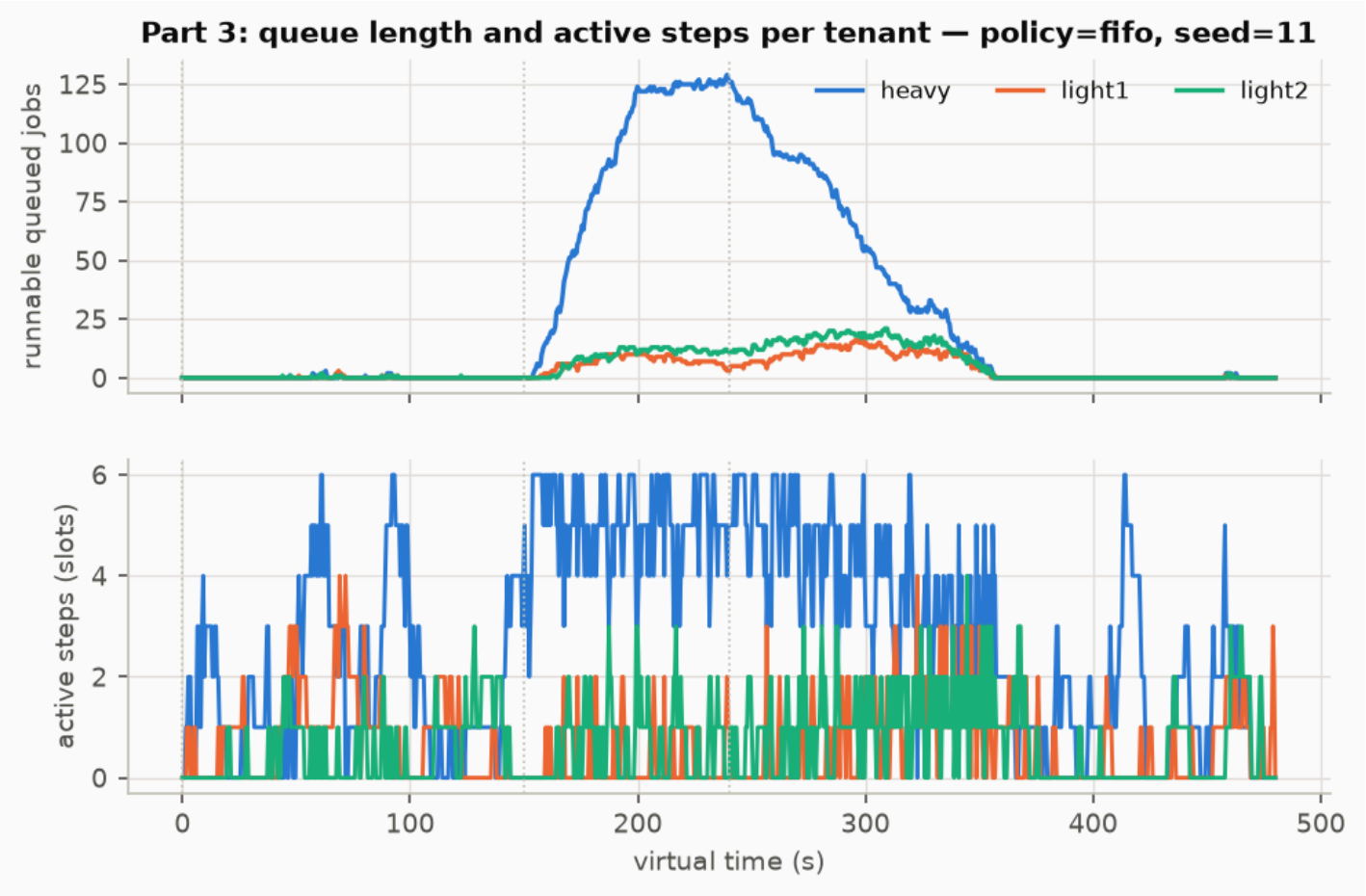
Доля выполненной работы во время перегрузки (окно burst 150–240 с, все seed)

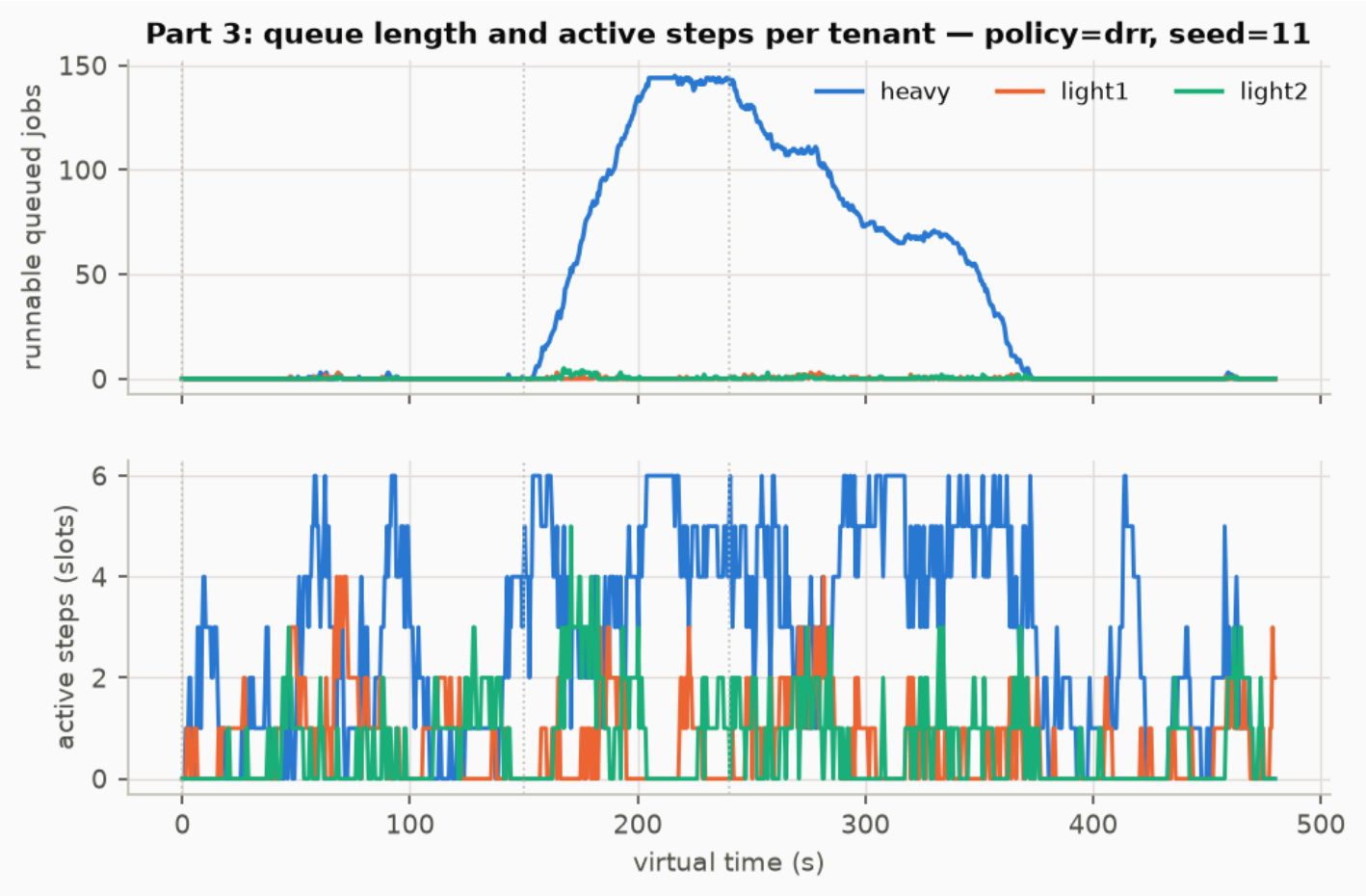
	heavy	light1	light2
спрос (номинальная работа заявок)	87.3%	5.5%	7.2%
вес / квота	33.3%	33.3%	33.3%
fifo	85.8%	6.0%	8.3%
drr	75.5%	11.2%	13.3%
drr_iso	62.4%	16.2%	21.4%

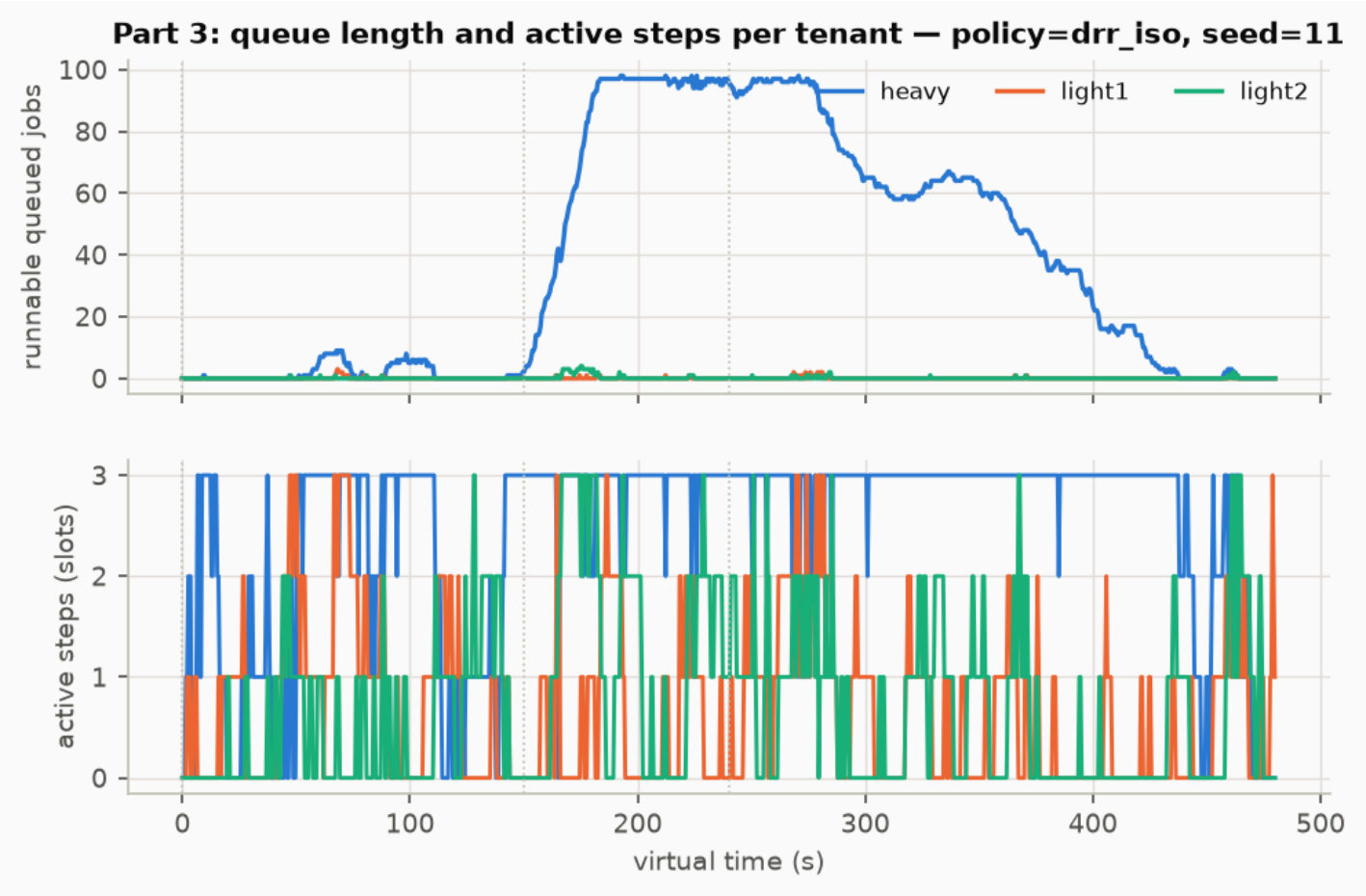


Длина очереди и активные шаги во времени


Part 3: queue length and wait of the oldest ready step (seed=11)
queued (all tenants)
150
100
50
0
fifo
drr
drr_iso
oldest ready step: wait since ready (s)
30
20
10
0
0
100
200
300
400
500
virtual time (s)


Part 3: queue length and active steps per tenant — policy=fifo, seed=11
runnable queued jobs
heavy
light1
light2
active steps (slots)
virtual time (s)


Part 3: queue length and active steps per tenant — policy=drr, seed=11
runnable queued jobs
heavy
light1
light2
active steps (slots)
virtual time (s)



Интерпретация

- **Изоляция.** В FIFO burst пользователя `heavy` заполняет общую очередь, и короткие задачи лёгких пользователей стоят за ней — их p95 ожидания вырастает до десятков секунд и часть их задач истекает или отклоняется. DRR выбирает следующий шаг по кругу между tenant, поэтому лёгкие пользователи, чей спрос ниже их доли (33 %), получают всё, что просят, а перегрузка ложится на виновника:
- light1: short p95 59.08 → 1.24 с, long p95 110.94 → 3.79 с (FIFO → DRR);
- light2: short p95 59.05 → 1.47 с, long p95 111.29 → 4.20 с (FIFO → DRR);
- heavy: short p95 59.12 → 59.66 с, long p95 117.01 → 161.93 с.
- **Throughput vs latency vs isolation.** `fifo` и `drr` work-conserving: загрузка слотов в burst 98.3% и 98.2%, DRR перераспределяет ожидание (и истечения) в сторону виновника перегрузки; число успешных job у DRR даже немного выше, т.к. короткие задачи лёгких пользователей не истекают. Throughput в job/с при одинаковой загрузке слотов не одинаков: в окне burst у `fifo` завершилось 317 job (из них long — 63), у `drr` — 357 (long — 97). Часть слот-времени уходит на шаги job, которые затем истекают по дедлайну (столбец «доля слот-времени…»: 6.0% у `fifo`, 5.3% у `drr`, 7.8% у `drr_iso`) — это работа, потерянная из-за перегрузки; остальная разница связана с тем, какие job успевают завершиться внутри окна, и отдельно не раскладывалась. `drr_iso` не даёт `heavy` занять больше 3 из 6 слотов даже когда остальные простаивают: загрузка слотов в burst 77.5%, throughput в burst 0.656 против 0.793 job/с у `drr` — плата за гарантированный запас ёмкости и за то, что лёгкие пользователи вообще не получают отказов (все 511 отказов `drr_iso` — у `heavy`). Квота очереди на tenant дополнительно защищает лёгких пользователей от отказов `queue_full`, которые в `fifo` / `drr` вызывает чужой burst (общий `max_queue_size`).
- **Чтение таблиц.** p95 ожидания `heavy` short ≈ 59–60 с во всех политиках — это цензурирование дедлайном 60 с (задачи истекают в очереди, см. TIMED_OUT); «e2e p50 успешных» считается только по выжившим задачам (survivorship), поэтому у `heavy` он мал при большом ожидании. Возраст старейшей job считается от её создания до последнего выхода из очереди (старт последнего шага или истечение в очереди); он ограничен дедлайном (60 с short / 200 с long) и у DRR больше, чем у FIFO: FIFO старит всех одинаково, DRR намеренно держит в очереди именно backlog `heavy`. Отдельный столбец «ожидание старейшего готового шага» (и график ниже) — возраст в очереди от момента готовности очередного шага; он сбрасывается после каждого шага, поэтому меньше.
- **Длительная перегрузка.** Admission control ограничивает очередь (429/REJECTED с причиной), дедлайны снимают job, которые не успеют, поэтому очередь и возраст старейшей job ограничены сверху, а не растут бесконечно; после окончания burst очередь рассасывается (фаза P3). Цена: почти все истёкшие job истекли в состоянии QUEUED *между* шагами, уже выполнив часть работы (1867 из 1867 истёкших по всем политикам успели исполнить хотя бы один шаг), — это и есть потерянное слот-время из сводной таблицы. Ранний отказ при admission дешевле, чем позднее истечение.
- **Планирование на границах шагов и ожидание коротких job.** После каждого шага job возвращается в очередь (`RUNNING → QUEUED`, `ready_at = now`) и заново конкурирует за слот. Поэтому шаг, стоящий первым в очереди, ждёт не всю 10-секундную long-job, а лишь освобождения любого слота — не дольше самого длинного шага (≤ 1.5 вирт. с в этой нагрузке; с реальной LLM шаг ограничен только `step_timeout_s` = 30 с). Но заходов столько, сколько шагов (медиана у успешных short — 6, у long — 9), и при backlog каждое повторное ожидание включает очередь перед job: под FIFO суммарное ожидание короткой job растёт как «число шагов × длина очереди» (p50 ожидания `heavy` short — десятки секунд, см. таблицу). DRR ограничивает это для лёгких пользователей, но не для перегружающего tenant. Цена шага как единицы планирования — лишняя транзакция claim/complete на каждый шаг (см. накладные расходы выше).

- **Aging.** Внутри tenant priority = 1 обгоняет priority = 0, но задача с низким приоритетом, прождавшая 10 с, получает +1 и не голодает (`tests/test_scheduling.py::test_aging_lets_old_low_priority_job_overtake`; измерения под нагрузкой — таблица «Ожидание по приоритету» выше).
- **Лимиты.** По трассе `step_runs` (интервалы исполнения) максимальная одновременность ни разу не превысила global/per-tenant лимит (столбцы «max … / лимит»; то же проверяют автотесты).
- **Ожидание approval не занимает слоты** — подтверждено автотестом `tests/test_timers_and_slots.py::test_waiting_job_does_not_occupy_execution_slot` (при `global_concurrency=1` вторая job полностью выполняется, пока первая ждёт approval) и E2/E6 (0 открытых `step_runs` в ожидании).

Approval под нагрузкой (ожидание не занимает слоты)

Тот же поток заявок, те же seeds [11, 12, 13, 14, 15] и ресурсы, политика `drr`; доля job, требующих approval, — 50% (выбор по seed, поток заявок не меняется); симулированный «человек» отвечает через 20 вирт. с после входа job в `WAITING_EVENT` (ожидание считается в дедлайн). Базовая строка — прогоны `drr` из основной таблицы. Сырые данные: `results/raw/loadappr_*` (`--approval-share 0.5 --approval-delay 20 --prefix loadappr_`).

вариант	принято	требуют approval	дошли до ожидания approval	SUCCEEDED	TIMED_OUT	jobs/s	загрузка слотов в burst	p95 ожидания в очереди light1 short, с	max одновременно в WAITING_EVENT	max активных шагов / лимит	моменты с ожидающими job, когда заняты все слоты	шагов job во время её же ожидания
drr, без approval (основной прогон)	2742	0	0	2085	657	0.861	98.2%	1.24 (N = 349)	0	6 / 6	–	0
drr, approval у 50% job	2743	1365	1102	2051	692	0.814	98.3%	1.37 (N = 343)	28	6 / 6	2308 из 4821	0

Ожидание approval не держит слот: одновременно в `WAITING_EVENT` находилось до 28 job, а активных шагов никогда не было больше лимита; в 48% моментов с ожидающими job все слоты были заняты *другой* работой (в остальные моменты очередь просто была короче лимита), и ни один шаг job не исполнялся, пока она ждала approval (последний столбец). Загрузка слотов в burst при этом та же, что без approval (98.3% против 98.2%). Цена ожидания — в истечениях и throughput: время ожидания входит в дедлайн, поэтому TIMED_OUT 692 против 657, а job, ожидающие человека, завершаются позже.

Part 3 на Linux (Docker-контейнер, те же seed и поток заявок)

Окружение: Linux-6.18.58.1-microsoft-standard-WSL2-x86_64-with-glibc2.41, Python 3.13.18; seeds [11, 12, 13, 14, 15], по одному прогону каждой политики на seed, масштаб времени 0.1. Сырые данные отдельно — `results/raw_linux/load_*`; с Windows-прогоном не смешиваются.

политика	прогонов	заявок	отклонено (по user)	SUCCEEDED	TIMED_OUT (по user)	jobs/s	загрузка слотов в burst	p95 ожидания light1 short, с	p95 ожидания heavy short, с	доля работы в burst heavy / light1 / light2	max global / лимит	max на tenant / лимит
fifo	5	2994	356 (heavy: 304, light1: 27, light2: 25)	2088	550 (heavy: 468, light1: 41, light2: 41)	0.862	98.9%	59.07 (N = 340)	59.11 (N = 1178)	85.6% / 6.0% / 8.4%	6 / 6	6 / 6
drr	5	2994	229 (heavy: 196, light1: 16, light2: 17)	2127	638 (heavy: 638)	0.878	98.9%	1.19 (N = 348)	59.66 (N = 1255)	76.0% / 10.4% / 13.5%	6 / 6	6 / 6
drr_iso	5	2994	503 (heavy: 503)	1908	583 (heavy: 583)	0.788	76.3%	0.92 (N = 362)	59.68 (N = 1026)	63.8% / 15.3% / 20.9%	6 / 6	3 / 3

Выводы Part 3 на Linux те же: DRR снижает p95 ожидания лёгких пользователей до единиц секунд, перегрузка (отказы и истечения) концентрируется у `heavy`, лимиты параллельности не превышаются, `drr_iso` жертвует загрузкой слотов ради изоляции.

6. Real LLM vs Simulated

Simulated (данные этого отчёта)

метрика	SimulatedLLM
LLM-вызовов на job (baseline)	2.0
токенов на job (prompt+completion)	348.2
стоимость на job, USD (иллюстративная цена из конфига)	0.000119
латентность LLM-вызова	задана конфигом: 1.0 с (Part 1)
время job baseline p50, с	2.96

Real LLM run

> **Статус: ЗАПУЩЕН** (данные из `results/real_llm/`, не смешиваются с симуляцией).

метрика	Real LLM (Ollama, локально)
модель / endpoint	qwen3.5:4b / http://localhost:11434/v1
job: итог	SUCCEEDED: 3
время job end-to-end, с (min–max)	91.76–95.39 (N = 3)
LLM-вызовов (ok / всего)	9 / 9
латентность LLM-вызова p50 / max, с	21.08 / 35.07 (N = 9)

метрика	Real LLM (Ollama, локально)
токенов prompt / completion	1382 / 931
стоимость, USD	0.000766 — условная (цена из конфига; локальная модель, реальных затрат нет)
решения агента `decide` (режим llm): действие × источник	request_approval/llm: 3
ответы модели в `decide` (уникальные, до 30 символов)	«REQUEST_APPROVAL»
recovery baseline: N / итог / время job p50, с / повторных LLM	5 / SUCCEEDED: 5 / 23.75 / 1
kill во время LLM: N / итог / resume после spawn p50, с	5 / SUCCEEDED: 5 / 2.62
kill во время LLM: повторных вызовов / доп. токены прерванных вызовов (подтверждено / верхняя граница) / перевыполнено шагов с чекпоинтом	5 / 0 / ≤1498 / 0
крэш после publish: N / итог / публикаций физ. / лог. / дублей / resume p50, с	5 / SUCCEEDED: 5 / 10 / 5 / 0 / 2.60

В основном прогоне 3 job стартуют одновременно, а локальная Ollama на CPU обрабатывает запросы по очереди (в `results/real_llm/calls.jsonl` каждый следующий вызов завершается вплотную к предыдущему), поэтому латентность вызова и время job включают ожидание в очереди модели; в сценариях восстановления job одна, отсюда меньшее время baseline.

Настоящая transient-ошибка. В сценариях восстановления 1 вызов LLM завершился ошибкой провайдера — Ollama вернула HTTP 500 «model runner has unexpectedly stopped» (процесс модели упал, по-видимому, из-за нехватки памяти); рантайм классифицировал её как transient, повторил шаг по политике ретраев (backoff, та же попытка шага +1), и job завершилась SUCCEEDED. Поэтому в baseline, где мы ничего не ломали, есть повторный вызов — это ретрай реальной ошибки, а не след восстановления.

Решения модели. В шаге `decide` модель выбирала: request_approval (llm): 3. Политика по правилам при `min_docs = 5` и 3 прочитанных документах дозапросила бы документы (`call_tool`); модель в 3 из 3 случаев вместо этого сразу отдала черновик человеку (`request_approval`). Это допустимое действие (эскалация к человеку разрешена всегда), поэтому ограничения не вмешались; качество такого выбора задание не оценивает. Ни одного нераспознанного ответа не было.

Чем реальная LLM отличается от симулятора (ожидаемо, проверить по таблице): латентность не детерминирована; ошибки 429/5xx/таймауты классифицируются как transient и ретраятся; прерванный крэшем вызов почти наверняка оплачен провайдером (верхняя граница доп. токенов становится реальной ценой); текст черновика при повторе может отличаться, но это не ломает идемпотентность публикации — тело собирается из *сохранённых* результатов шагов, а повтор `draft` после чекпоинта невозможен.

Запросы к реальной LLM идут с `temperature = 0` (`src/agentrt/llm.py`), поэтому в сценариях восстановления с одним номером повтора (одинаковые документы и промпты) число completion-токенов совпадает между сценариями (см. `recovery_calls_real.jsonl`); между тремя job основного прогона промпты различаются, и токены разные (`calls.jsonl`). Для прерванного вызова в журнале записан только prompt (ответа не было).

Real LLM run: сервер ITMO (qwen3.8-27b)

> **Статус: ЗАПУЩЕН** (данные из `results/real_llm_server/`; не смешиваются ни с симуляцией, ни с прогоном на qwen3.5:4b).

метрика	Real LLM (сервер ITMO)
модель / endpoint	qwen3.8-27b / <внутренний адрес сервера университета, доступ через VPN>/v1
job: итог	SUCCEEDED: 3
время job end-to-end, с (min–max)	6.24–6.36 (N = 3)
LLM-вызовов (ok / всего)	6 / 6
латентность LLM-вызова p50 / max, с	1.72 / 1.98 (N = 6)
токенов prompt / completion	702 / 910
стоимость, USD	не считалась (тариф сервера неизвестен; цены в прогоне = 0)
recovery baseline: N / итог / время job p50, с / повторных LLM	1 / SUCCEEDED: 1 / 3.06 / 0
kill во время LLM: N / итог / resume после spawn p50, с	1 / SUCCEEDED: 1 / 1.31
kill во время LLM: повторных вызовов / доп. токены прерванных вызовов (подтверждено / верхняя граница) / перевыполнено шагов с чекпоинтом	1 / 0 / ≤299 / 0
крэш после publish: N / итог / публикаций физ. / лог. / дублей / resume p50, с	1 / SUCCEEDED: 1 / 2 / 1 / 0 / 1.37

Этот прогон сделан раньше, с одним повтором на сценарий и решением `decide` по правилам (иллюстрация, а не статистика); повторить его с большим числом повторов не удалось — сервер доступен только через VPN университета. Латентность и время job нельзя напрямую сравнивать с прогоном на Ollama: другая модель и другое железо (CPU против сервера университета). Столбцы `cost_*` в `results/real_llm_server/recovery_runs_real.csv` посчитаны подпрограммой восстановления по иллюстративным ценам из конфига по умолчанию и реальной стоимостью не являются.

7. Ответы на вопросы задания

1. Где хранится состояние? В одном файле SQLite (WAL) рантайма: `jobs` (состояние, версия, lease, следующий шаг, счётчики), `step_results` (чекпоинты шагов), `events` / `event_receipts` (inbox), `outbox`, `timers` (дедлайны), `scheduler` (DRR), `transitions` (журнал, пишется триггером), `step_runs` (трасса исполнения и занятые слоты). В памяти воркера нет ничего, что нельзя потерять: после SIGKILL всё восстанавливается из БД. Внешний сервис публикаций хранит свои публикации в своей БД.

2. Что при сбое между обновлением состояния и публикацией следующего события? Такого окна нет: обновление состояния и запись сообщения в `outbox` (`approval.requested`, `job.<terminal>`) — одна транзакция; следующий шаг тоже не «событие в брокере», а поле `next_step` в той же строке. Если

процесс упал после коммита, но до доставки, relay при рестарте найдёт строку `delivered_at IS NULL` и доставит её (at-least-once); получатель дедуплицирует по `message_id` (`Idempotency-Key`). Если упал до коммита — не было ни нового состояния, ни сообщения; шаг повторится после истечения lease. Окно, которое транзакцией закрыть нельзя, — внешний вызов: сервис публикаций уже закоммитил публикацию, а наш чекпоинт шага `publish` ещё нет. Его закрывает не транзакция, а идемпотентность: повтор шага идёт с тем же `Idempotency-Key` и получает прежнюю публикацию (Part 1, сценарии 3a/3b); если же job тем временем отменена/истекла, отзыв публикации записывается в `outbox` вместе с терминальным переходом (вопрос 4).

3. Какие операции могут повторяться и как ограничены последствия? Повторяется только незакоммиченная попытка шага: LLM-вызов, tool-вызов, publish, а также доставка outbox. Ограничения: (а) чекпоинт — шаг с сохранённым результатом не перевыполняется; (б) `max_attempts` на шаг (крэш тоже считается попыткой — защита от poison pill), backoff с jitter, `step_timeout`, `max_steps`, `max_tokens`; (в) внешний side effect идемпотентен (`Idempotency-Key`), поэтому повтор publish даёт 0 дублей логических публикаций; (г) повтор LLM стоит денег — измерено как доп. вызовы/токены (Part 1), с верхней границей для вызовов с неизвестным исходом; (д) фенсинг по `lease_token` не даёт «зомби»-воркеру закоммитить поздний результат.

4. Как разрешаются гонки cancel / deadline / completion? Побеждает первая закоммиченная транзакция перехода в терминальное состояние (SQLite сериализует писателей, переход — CAS по `version` с `state NOT IN terminal`, триггер запрещает менять терминальные строки). При «одновременности» действует приоритет CANCELLED > TIMED_OUT > approval > step completion: cancel не проверяет дедлайн; approval и коммит шага проверяют `now ≥ deadline` и вместо себя коммитят TIMED_OUT; коммит шага фенсится lease. Поздние результаты отбрасываются, поздние события получают `ignored_terminal`. Если publish мог пройти, а job отменена/истекла/упала, выполняется компенсация — идемпотентный retract публикации; он записывается в `outbox` в той же транзакции, что и терминальный переход, поэтому переживает падение воркера (автотест `test_publication_retracted_durably_when_job_cancelled_after_worker_crash`). E5: 0 нарушений инвариантов и 0 нарушений правила по каждой job на всех повторах (таблица в E5).

5. Что обеспечивает справедливость и что при длительной перегрузке? DRR по tenant на границах шагов (веса, quantum в секундах работы) + aging внутри tenant; опционально жёсткие per-tenant лимиты слотов и квоты очереди. При длительной перегрузке очередь ограничена admission control (429/REJECTED с причиной и метрикой), дедлайны снимают задачи, которые не успеют (TIMED_OUT, в т.ч. из очереди), DRR не даёт одному tenant вытеснить остальных. `fifo` и `drr` work-conserving — в burst слоты заняты почти полностью (столбец «загрузка слотов в burst» в Part 3), а DRR только перераспределяет ожидание, отказы и истечения в сторону tenant-источника перегрузки; `drr_iso` сознательно не work-conserving: жёсткий per-tenant лимит оставляет часть ёмкости свободной ценой throughput, зато лёгкие пользователи не получают ни отказов, ни истечений (Part 3). Throughput в job/с не равен номинальной ёмкости из Part 3 (она посчитана для смеси заявок): он зависит от того, какая доля выполненных задач — long; отдельно эта разница не анализировалась.

6. Какие гарантии от инфраструктуры, а какие из кода? Инфраструктура (SQLite): атомарность, сериализуемость писателей, долговечность коммита (WAL), уникальные индексы и триггеры. Код: модель состояний и CAS, lease/heartbeat/reaper/recovery, фенсинг, ретраи и классификация ошибок, дедуп событий (через PK), outbox relay, durable-таймеры, admission control, планировщик, компенсация, идемпотентный ключ публикации. Внешний сервис отвечает за собственную дедупликацию по ключу. Подробная таблица — README, раздел «Гарантии инфраструктуры vs кода приложения».

8. Ограничения и честные оговорки

- **Real LLM: два прогона, небольшая статистика.** Основной прогон — локально, `qwen3.5:4b` (Ollama, CPU), `reasoning_effort=none` (без него модель тратит токены на рассуждение и возвращает пустой `content`): 3 job с выбором действия через LLM (`decide_mode = llm`, `min_docs = 5`) и по 5 повтор(ов) трёх сценариев восстановления (baseline, kill во время LLM-вызова, крэш после публикации). Таймаут ожидания воркера в сценарии восстановления 300 с из-за медленной локальной модели. Две самые первые попытки завершились HTTP 503 (неверное имя модели в `.env`; затем 503 до отключения рассуждения) и в отчёт не включены. Ещё одна попытка этого прогона (перед показанным) не состоялась: все вызовы модели завершились HTTP 500, локальной Ollama не хватило памяти для загрузки модели (`out-of-memory` при старте `llama-server`, пока на машине работали Docker и другие программы); её данные не включены, прогон повторён после освобождения памяти. Второй прогон — сервер университета, `qwen3.8-27b` (vLLM, доступ только через VPN, токен только в локальном `.env`): 3 job, решение по правилам, по одному повтору сценариев; повторить его не удалось (сервер был недоступен). Стоимость сервера не считалась (тариф неизвестен); цена для Ollama условная. Влияет ли `LLM_REASONING_EFFORT=none` на серверную модель, не проверялось. Результаты двух моделей и Simulated не смешиваются.
- **Один узел, один писатель.** SQLite сериализует все записи; масштабирование — несколько процессов на одной машине (лимиты проверяются в транзакции и для них верны), но не кластер. Узкое место — частота транзакций (claim + complete на каждый шаг + heartbeats).
- **Основная ОС — Windows; Linux — контейнер и CI.** Основные таблицы Part 1–3 получены на Windows 10 («SIGKILL» там — `Popen.kill()` → `TerminateProcess`, без обработчиков/`finally`). Все три части и автотесты повторены в Linux-контейнере с настоящим POSIX SIGKILL (`results/raw_linux/`, блоки «на Linux») — это Docker Desktop с ядром WSL2 на том же железе. На **отдельной** Linux-машине (GitHub Actions, `ubuntu-24.04`, `.github/workflows/ci.yml`) на каждый push гоняются все автотесты, Part 1 и Part 2 с автоматической проверкой гарантий; сборка падает при нарушении. Числа CI в таблицы отчёта не переносятся (они на странице прогона и в артефакте `ci-results`); Part 3 в CI не гоняется — общий раннер с плавающей ёмкостью исказил бы времена.
- **`synchronous=NORMAL`**: гарантия durable — против падения процесса (это и проверялось), но не против потери питания/краха ОС для последних транзакций (`AGENTRT_SYNC=FULL` закрывает это ценой латентности; не измерялось).
- **Масштабированное время** в Part 3 (1 вирт. с = 0.1 с): накладные расходы рантайма (транзакции, поллинг) не масштабируются и в вирт. секундах выглядят больше (оценка приведена). Абсолютные значения — для этой машины и этого масштаба; сравнение политик корректно, т.к. условия одинаковые.
- **Статистика.** Part 1: 10 повторов на сценарий; Part 2: 300 повторов гонки E5; Part 3: 5 прогонов (seed) на политику — разброс p95 по seed показан, доверительные интервалы не считались. Для `light*` × `long` и priority = 1 наблюдений меньше (см. N).
- **Неопределённость внешних вызовов** не устранима на стороне клиента: прерванный LLM-вызов может быть оплачен; мы даём верхнюю оценку (prompt + max_output_tokens), а не точное значение.
- **Окно публикации при cancel.** Отмена во время `publish` не может «отозвать» уже отправленный запрос; инвариант «нет живой публикации у не-SUCCEEDED job» конечный (eventual): публикация существует до отзыва (до закрытия run и следующего прохода relay; при падении воркера — до истечения lease). Долгая недоступность сервиса публикаций не проверялась: отзыв повторяется без ограничения числа попыток.
- **Мгновенное восстановление по имени воркера** (`recover_own`) предполагает, что прежний процесс с тем же `--name` мёртв; это не проверяется (нет проверки pid). Два живых воркера с одним именем — ошибка конфигурации: фенсинг по `lease_token` не даст закоммитить чужой результат, но шаг (и LLM-вызов) может выполниться дважды.

- **Approval под нагрузкой проверен только для** `drr` и одной доли/задержки (50 %, 20 вирт. с, отдельная таблица в Part 3); основные таблицы Part 3 — без approval, чтобы сравнение политик не зависело от поведения «человека».
- **Цены токенов** в конфиге иллюстративные (не прайс-лист провайдера); стоимость = токены × цена из конфига.
- **Межпроцессных уведомлений нет**: воркер узнаёт о новых событиях/заявках из других процессов поллингом (`poll_interval_s`), это входит в латентность «event → next step».
- **Outbox без dead-letter.** Недоставленные уведомления повторяются с ограниченной паузой (≤ 8 с), но без предела числа попыток; при долго недоступном получателе строки копятся в `outbox` (видны по `attempts`, `last_error`), отдельной очереди «мёртвых» сообщений нет. На состояние job это не влияет.
- `docker-compose.yml` **проверен сквозным сценарием**, а не нагрузкой: `scripts/demo_docker.ps1` поднимает publisher, api и worker, через HTTP API проверяет идемпотентное создание, SIGKILL воркера (`docker compose kill`) с автоматическим продолжением, дубль события и пересоздание всех контейнеров во время ожидания approval (состояние, дедлайн и `approval_id` сохраняются в именованном томе). Отчётные эксперименты в compose не запускались.
- **Воспроизводимость с нуля проверена** на той же машине: клон из Git → новое окружение из `requirements.txt` → `scripts/run_all_experiments.ps1` (≈ 14 мин) — тесты зелёные, выводы всех трёх частей совпали; абсолютные времена и доли исходов E5 отличаются (см. E5). В другой ОС/на другом железе не проверялось.
- **Решение агента через LLM** (`decide_mode = llm`) использовано в прогоне с реальной LLM (решения и их источник — в таблице Real LLM); симуляционные эксперименты Part 1–3 идут на правилах — это сохраняет детерминизм и сравнимость политик планирования. Ограничения (бюджет tool-вызовов, обязательный approval) проверяются кодом, поэтому модель не может их нарушить, но качество её выбора не оценивалось (задание не оценивает качество ответов LLM).